Week 2 – Process 1

Operating Systems Ch 3 (Sections 3.1 – 3.3)

Korea University Sejong Campus, Department of Computer Science

Table of Contents

1. Process Concept (The Process, State, PCB, Threads)
2. Process Scheduling (Queues, Context Switch)
3. Operations on Processes (Creation, Termination)
4. Lab – fork(), exec(), wait()
5. Summary and Next Week Preview

Part 1

Process Concept

Silberschatz Ch 3.1

What Is a Process?

Process = a program in execution

- Early computers: only one program running at a time
- Modern computers: multiple programs loaded into memory and executed **concurrently**
- This evolution required stricter control and compartmentalization, giving rise to the **process** concept

> "A process is the unit of work in a modern computing system."

- From the OS perspective, both processes executing user code and processes executing OS code exist
- The CPU(s) are **multiplexed** among these processes

The Process – Memory Layout (Figure 3.1)

The memory layout of a process is divided into several **sections**:

```
max (high address)
← Temporary data during function calls (parameters, return address, local variables)
Stack
  ↓

  ↑
Heap
← Dynamically allocated memory at runtime (malloc, new)
Data
← Global variables
Text
← Executable code
0 (low address)
```

- **Text / Data** sections: **fixed** in size (do not change during program execution)
- **Stack / Heap** sections: grow and shrink **dynamically**

Dynamic Growth of Stack and Heap

Stack growth:

- Each time a function is called, an **activation record** (function parameters, local variables, return address) is pushed onto the stack
- When a function returns, the activation record is popped
- The stack grows from **high addresses toward low addresses** (downward)

Heap growth:

- When memory is dynamically allocated with `malloc()`, `new`, etc., the heap grows
- When freed with `free()`, `delete`, the heap shrinks
- The heap grows from **low addresses toward high addresses** (upward)

Role of the OS:

- Since the stack and heap grow toward each other, the OS must ensure they do **not overlap**

Memory Layout of a C Program (Detailed)

```c
#include <stdio.h>
#include <stdlib.h>

int x;            // uninitialized data (BSS)
int y = 15;       // initialized data

int main(int argc, char *argv[]) {   // argc, argv area
    int *values;   // local variable → stack
    int i;         // local variable → stack

    values = (int *)malloc(sizeof(int)*5);  // dynamic allocation → heap
    for (i = 0; i < 5; i++)
        values[i] = i;
    return 0;
}
```

You can check the section sizes with the `size` command:

text	data	bss	dec	hex
1158	284	8	1450	5aa

Program vs. Process

Distinction	Program	Process
Nature	**Passive** entity	**Active** entity
Form	Executable file stored on disk	Execution unit with a PC and associated resources
Conversion	Becomes a process when the executable is loaded into memory	

Key distinctions:

- The same program can run as multiple processes
  - Example: multiple users running the same web browser simultaneously
  - The text section is identical, but **data, heap, and stack are separate for each**

Process State (Figure 3.2)

A process changes its **state** during execution:

State	Description
New	The process is being created
Running	Instructions are being executed
Waiting	The process is waiting for some event (I/O completion, signal, etc.)
Ready	The process is waiting to be assigned to a processor
Terminated	The process has finished execution

Important: At any point in time, only **one process** can be in the running state on a single processor core.

Process State Transition Diagram (Figure 3.2)

```
                admitted                                exit
New ──────────────────→ Ready ──────────────────────────────→ Terminated
                         ↑    scheduler dispatch      │
                         │          ↓                 │
                         │        Running ────────────┘
                         │          │
    I/O or event         │          │ I/O or event wait
    completion           │          ↓
                         └──────── Waiting

                   ← interrupt ──┘
              (Ready ← Running)
```

Process State Transitions – Detailed Explanation

Transition	Cause
New → Ready	admitted – The OS approves process creation; memory allocation is complete
Ready → Running	scheduler dispatch – The CPU scheduler selects the process and assigns it to a CPU core
Running → Ready	interrupt – The time slice expires, causing the CPU to be preempted
Running → Waiting	I/O or event wait – An I/O request is made or a specific event is awaited
Waiting → Ready	I/O or event completion – The awaited I/O or event has completed
Running → Terminated	exit – The process finishes execution and terminates

- **Running → Waiting**: The process voluntarily transitions (e.g., requesting a disk read)

Process Control Block – PCB (Figure 3.3)

Each process is represented in the OS by a **PCB (Process Control Block)**.

- Also known as: **task control block**

```
┌──────────────────────────┐
│   Process state          │  ← new, ready, running, waiting, terminated
│   Process number (pid)   │  ← unique process identifier
│   Program counter        │  ← address of the next instruction to execute
│   Registers              │  ← CPU register values
│   Memory limits          │  ← memory management information
│   ...                    │
│   List of open files     │  ← I/O status information
└──────────────────────────┘
```

The PCB serves as the repository for all the data needed to **start or restart** a process.

Detailed Information Stored in the PCB

Field	Description
Process state	Current state (new, ready, running, waiting, terminated)
Program counter	Address of the next instruction to execute
CPU registers	Accumulator, index register, stack pointer, general-purpose registers, condition codes, etc.
CPU scheduling info	Process priority, scheduling queue pointers, other scheduling parameters
Memory-management info	Base/limit register values, page table or segment table
Accounting info	CPU time used, elapsed real time, time limits, account number, process number
I/O status info	List of allocated I/O devices, list of open files

PCB in Linux – task_struct

In the Linux kernel, the PCB is represented by the `task_struct` C structure.

- Location: `<include/linux/sched.h>`

```
long state;                     /* process state */
struct sched_entity se;         /* scheduling information */
struct task_struct *parent;     /* parent process */
struct list_head children;      /* list of child processes */
struct files_struct *files;     /* list of open files */
struct mm_struct *mm;           /* address space (memory management) */
```

- All active processes are managed via a **doubly linked list**
- The `current` pointer points to the `task_struct` of the currently running process

```
current→state = new_state;  // change the state of the current process
```

Thread Overview (Ch 3.1.4)

The process model discussed so far assumes a **single thread of execution**:

- A process can perform only one task at a time
- Example: in a word processor, you cannot type and run a spell checker at the same time

Thread = a unit of execution within a process

Modern operating systems allow **multiple threads of execution**:

- A single process can perform multiple tasks simultaneously
- **Parallel execution** is possible on multicore systems

Advantages and Examples of Threads

Multithreaded word processor example:

- Thread 1: Manages user input
- Thread 2: Runs the spell checker
- Both can run simultaneously!

Threads and the PCB:

- In systems that support threads, the PCB is extended to **include information for each thread**
- System-wide changes are needed to support threads

What threads of the same process share:

- Text section, Data section, Heap, open files, signals, etc.

What is independent per thread:

- Program counter, register set, stack

> Detailed coverage of threads is in **Ch 4**.

Part 2

Process Scheduling

Silberschatz Ch 3.2

Why Is Process Scheduling Needed?

Goal of multiprogramming:

- Maximize **CPU utilization** by ensuring some process is always running

Goal of time sharing:

- Frequently switch the CPU core among processes so users can **interact** with each program

-> **Process Scheduler**: Selects which process to run on a CPU core from among the available processes

- Single core: only one process can be running at a time
- Multicore: multiple processes can be running simultaneously
- If the number of processes exceeds the number of cores, the rest must wait

Degree of Multiprogramming and Process Types

Degree of Multiprogramming:

- The number of processes currently in memory

Classification by process type:

Type	Characteristics
I/O-bound process	Spends more time doing I/O than computation; issues I/O requests frequently
CPU-bound process	Spends more time doing computation than I/O; issues I/O requests infrequently

- For efficient scheduling, it is important to have a proper mix of I/O-bound and CPU-bound processes
- Only I/O-bound processes -> ready queue empties, CPU idles
- Only CPU-bound processes -> wait queue empties, I/O devices idle

Scheduling Queues (Figure 3.4)

When a process enters the system, it is placed in a **scheduling queue**.

Ready Queue:

- A queue of processes waiting for the CPU
- Typically implemented as a **linked list**
- Queue header: contains a pointer to the first PCB
- Each PCB: contains a pointer field to the next PCB

Wait Queue:

- A queue of processes waiting for a specific event (e.g., I/O completion)
- A separate wait queue may exist for each device

```
Ready Queue:  [header] → [PCB₂] → [PCB₃] → ...
Wait Queue:   [header] → [PCB₇] → [PCB₁₄] → [PCB₆] → ...
```

Queueing Diagram (Figure 3.5)

```
                    ┌──────────────┐
                    │ Ready Queue  │──→ [CPU] ──→ (process terminates)
                    └──────────────┘                │
                          ↑                         │
                          │                         ├── I/O request → [I/O Wait Queue] → I/O complete → (Ready Queue)
                          │                         │
                          │                         ├── time slice expired → (Ready Queue)
                          │                         │
                          │                         ├── child process created → [Child Termination Wait Queue]
                          │                         │                              → child terminates → (Ready Queue)
                          │                         │
                          │                         └── waiting for interrupt → [Interrupt Wait Queue]
                          │                                                   → interrupt occurs → (Ready Queue)
                          └──────────────────────────────────────────────────────────────────────┘
```

The process repeats this cycle until it terminates. Upon termination, it is removed from all queues, and its PCB and resources are deallocated.

CPU Scheduling (Ch 3.2.2)

Role of the CPU Scheduler:

- Selects one of the processes in the ready queue and assigns it to a CPU core

Scheduling frequency:

- I/O-bound processes: execute for only a few milliseconds before waiting for I/O
- CPU-bound processes: execute longer, but the scheduler forcibly reclaims the CPU
- Typically, the CPU scheduler runs at least once every **100 ms**

Swapping (an intermediate form of scheduling):

- Moves a process from memory to disk (**swap out**) and later reloads it (**swap in**)
- Reduces the degree of multiprogramming to relieve memory pressure
- Only necessary when memory is **overcommitted**
- Covered in detail in Ch 9

Context Switch Concept (Figure 3.6)

Context:

- Represented in the PCB of the process
- Includes CPU register values, process state, memory management information, etc.

Context Switch:

- The operation of switching a CPU core to a different process
- **State save** of the current process + **state restore** of the new process

When an interrupt occurs:

1. The system must save the context of the current process
2. After handling, it restores that context to resume the process

Context Switch Process – Detailed (Figure 3.6)

```
Process P0          Operating System          Process P1
__________          ________________          __________

executing
     |
interrupt or
system call
     |
     └──────→  save state into PCB0
                     |
               reload state from PCB1
                     |
                     └──────────────→  executing
                                            |
                                       interrupt or
                                       system call
                                            |
                     ┌──────────────── save state into PCB1
                     |
               reload state from PCB0
                     |
     ┌───────────────┘
     |
executing
```

Overhead of Context Switching

Context switch time = pure overhead

- The system performs **no useful work** during the switch
- Typical speed: **several microseconds**

Factors affecting speed:

- Memory speed
- Number of registers that must be copied
- Availability of special instructions (e.g., instructions that load/store all registers at once)

Hardware support:

- Some processors provide **multiple register sets**
- In such cases, a context switch can be achieved simply by changing the current register set pointer
- If the number of active processes exceeds the number of register sets, memory copying is required again

Multitasking on Mobile Systems

iOS:

- Early iOS: no multitasking for user apps (only the foreground app runs; the rest are suspended)
- Starting with iOS 4: limited multitasking (1 foreground + multiple background)
- Later versions: richer multitasking support as hardware improved
- iPad: simultaneous execution of 2 foreground apps via split-screen

Android:

- Supported multitasking from the beginning, with no restrictions on background app types
- Uses a **Service** (a separate app component) for background processing
  - Example: a music streaming app continues audio delivery via a service even when moved to the background
  - Services have no UI and use less memory
  - Even if a background app is suspended, its service continues to run

Part 3

Operations on Processes

Silberschatz Ch 3.3

Process Creation (Ch 3.3.1)

A process can create several new processes during its execution.

Term	Description
Parent process	The process that creates another
Child process	The newly created process
Process tree	A tree structure formed by parent-child relationships
Process ID (pid)	A unique identifier for each process (typically an integer)

- The pid is used as an **index** within the kernel to access various attributes of the process
- Most operating systems including UNIX, Linux, and Windows use pids

Linux Process Tree (Figure 3.7)

```
                      systemd
                      pid = 1
                  /     |      \
                /       |        \
          logind      python      sshd
        pid=8415    pid=2808    pid=3028
            |                       |
          bash                    sshd
        pid=8416                pid=3610
         /    \                     |
       ps     vim                  tcsh
    pid=9298  pid=9204          pid=4005
```

- **systemd** (pid = 1): The **root parent** of all user processes
  - The first user process created during system boot
  - Creates processes for additional services such as web servers, SSH servers, etc.
- In traditional UNIX, **init** (pid = 1) performed this role
- Recent Linux distributions have replaced init with **systemd**

Process Creation – Resources and Execution Options

How a child process obtains resources:

1. Obtains resources directly from the OS
2. Receives a subset of the parent process's resources
   - The parent distributes or shares resources with its children
   - Resource limits prevent excessive child process creation

Execution options:

1. The parent continues executing **concurrently** with the child
2. The parent **waits** until the child terminates

Address space options:

1. The child is a **duplicate** of the parent – same program and data
2. A **new program** is loaded into the child

Process Creation in UNIX/Linux

In UNIX, a new process is created with the `fork()` system call.

How fork() works:

1. Creates a new process as a **copy of the calling process's address space**
2. **Both** the parent and child continue execution from the instruction after fork()

Distinguishing parent from child via fork()'s return value:

Return Value	Meaning
0	Child process (newly created process)
Positive (> 0)	Parent process (the child's pid is returned)
-1	fork failed (error)

- The child inherits the parent's **privileges**, **scheduling attributes**, **open files**, etc.

exec() After fork()

exec() system call: **replaces** the process's memory space with a new program

```
fork() → child process created (copy of parent)
         |
    child: calls exec()
         |
    child's memory image replaced with the new program
    (the original program is destroyed)
```

- exec() loads a binary file into memory and begins execution
- If exec() succeeds: it **does not return** to the original code (memory has been overwritten)
- If exec() fails: control is returned (error handling is possible)
- If the child does not call exec(): it continues executing the copy of the parent's code

fork() Code Example (Figure 3.8)

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) {          /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) {    /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else {                  /* parent process */
        /* parent waits for child to complete */
        wait(NULL);
        printf("Child Complete");
    }
```

fork() + exec() + wait() Flow Diagram

```
Parent Process
    │
    ├── fork() called
    │       │
    │       ├── [Child process created]
    │       │       │
    │       │       ├── pid == 0 (child)
    │       │       │       │
    │       │       │       └── execlp("/bin/ls", "ls", NULL)
    │       │       │               → executes ls command
    │       │       │               → terminates via exit()
    │       │       │
    │       ├── pid > 0 (parent)
    │       │       │
    │       │       └── wait(NULL)  ← waits for child to terminate
    │       │               │
    │       │               └── prints "Child Complete"
    │       │
    │
    └── return 0
```

This is the fundamental operating principle of the UNIX shell!

Deep Dive into fork() Return Values

After fork(), the parent and child have the same code, but the return value differs:

```
pid_t pid = fork();
// At this point, there are 2 processes!

printf("pid = %d\n", pid);
```

Output (execution order is not guaranteed):

```
pid = 3456    ← parent: prints the child's actual pid
pid = 0       ← child: prints 0
```

Why was it designed this way?

- Parent: needs the child's pid for management purposes (wait, kill, etc.)
- Child: can obtain its own pid with `getpid()` and its parent's pid with `getppid()`

Process Creation – Windows Comparison

Windows: `CreateProcess()` **API**

Comparison	UNIX fork()	Windows CreateProcess()
Address space	**Duplicates** parent's address space	**Specifies and loads** a new program
Parameters	None (0)	**10+** required
Waiting for child	`wait()`	`WaitForSingleObject()`
Process info	pid (integer)	`PROCESS_INFORMATION` struct (handles + identifiers)

- Windows `STARTUPINFO` : specifies the new process's window size, appearance, standard I/O handles, etc.
- Windows `PROCESS_INFORMATION` : contains handles and identifiers for the created process and thread

Process Termination (Ch 3.3.2)

Normal termination:

- The process executes its last statement and requests deletion from the OS via the `exit()` system call
- A status value (typically an integer) is passed to the **waiting parent** (via `wait()`)
- The OS reclaims all of the process's resources (physical/virtual memory, open files, I/O buffers)

```
/* exit with status 1 */
exit(1);
```

Parent's wait() call:

```
pid_t pid;
int status;
pid = wait(&status);  // returns the terminated child's pid; stores exit status in status
```

Forced Termination by Parent (abort)

Situations where a parent process may **forcibly terminate** a child:

1. The child has **exceeded its allocated resources**
   - The parent needs a mechanism to inspect the child's state
2. The **task assigned to the child is no longer needed**
3. The **parent is terminating** and the OS does not allow orphan processes

Cascading Termination:

- A system where all children must be terminated when the parent terminates
- Parent terminates -> children terminate -> grandchildren terminate -> …
- Typically **initiated by the OS**
- Forcibly terminating a process in Linux: the `kill` command or the `kill()` system call
- In Windows: the `TerminateProcess()` API

Zombie Process

What is a zombie process?

- A child process that has **terminated (exit)**, but whose parent has **not yet called wait()**
- The process has finished executing, but its **process table entry remains**
  - Because the exit status information must be retained

Every process briefly passes through the zombie state upon termination!

- Typically remains a zombie for only a very short time
- When the parent calls `wait()`:
  - The zombie process's pid and process table entry are **released**

Problem:

- If the parent never calls `wait()`, zombies continue to accumulate, wasting system resources

Orphan Process

What is an orphan process?

- A child process left behind because its parent **terminated first** without calling `wait()`

Handling in UNIX/Linux:

- Traditional UNIX: The **init** process (pid = 1) becomes the new parent of orphan processes
- Modern Linux: **systemd** performs this role
  - Linux also allows processes other than systemd to inherit and manage orphan processes

Orphan process cleanup procedure:

1. The parent terminates
2. The orphaned child process is adopted by init/systemd
3. init/systemd periodically calls `wait()`
4. The orphan process's exit status is collected, and the process table entry is released

Zombie vs. Orphan Comparison

Distinction	Zombie Process	Orphan Process
State	Child has **terminated**	Child is **still running** (parent terminated first)
Cause	Parent has **not called wait()**	Parent **terminated first**
Resources	Occupies only a process table entry	Running normally
Resolution	Parent calls wait()	init/systemd adopts and manages

```
[Zombie Scenario]
Parent (running)
   └── Child (terminated)
       → wait() not called
       → zombie state
```

```
[Orphan Scenario]
Parent (terminated)
   └── Child (running)
       → adopted by init/systemd
       → continues normal execution
```

Android Process Hierarchy (Ch 3.3.2.1)

Mobile operating systems may need to terminate existing processes due to limited resources. Android determines the termination order based on an **importance hierarchy**:

Rank	Type	Description	Termination Priority
1	**Foreground process**	The app currently visible on screen and being interacted with by the user	Last to be terminated
2	**Visible process**	Not directly visible but performing an activity referenced by a foreground app	
3	**Service process**	In the background but performing an activity apparent to the user (e.g., music streaming)	
4	**Background process**	Performing an activity not visible to the user	

Chrome Browser's Multiprocess Architecture

Problem: In a tab-based web browser, if a web app in one tab crashes, the entire browser crashes

Chrome's solution: Multiprocess architecture

Process Type	Role	Count
Browser	UI management, disk/network I/O	1
Renderer	Web page rendering (HTML, JS, images)	1 per tab
Plug-in	Plug-in code execution (Flash, etc.)	1 per plug-in type

Advantages:

- Websites are **isolated** from each other – a crash on one site only affects that renderer
- Renderer processes run in a **sandbox** – restricted from disk/network I/O access
- Minimizes the impact of security vulnerabilities

Part 4

Lab – fork(), exec(), wait()

Lab: Basic Usage of fork()

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Before fork: pid = %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        /* child process */
        printf("Child: my pid = %d, parent pid = %d\n",
               getpid(), getppid());
    } else {
        /* parent process */
        printf("Parent: my pid = %d, child pid = %d\n",
               getpid(), pid);
        wait(NULL);
    }
```

Lab: Creating Multiple Children with fork()

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int i;

    for (i = 0; i < 3; i++) {
        pid = fork();
        if (pid == 0) {
            printf("Child %d: pid = %d, parent pid = %d\n",
                   i, getpid(), getppid());
            return 0;  // child exits here (important!)
        }
    }
    // parent: wait for all children to terminate
    for (i = 0; i < 3; i++) {
        wait(NULL);
    }
    printf("Parent: all children finished\n");
    return 0;
}
```

Lab: Running a New Program with exec()

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        /* child: execute ls -l command */
        printf("Child: about to exec ls -l\n");
        execlp("ls", "ls", "-l", NULL);

        /* the code below is never executed if exec() succeeds */
        perror("exec failed");
        return 1;
    } else if (pid > 0) {
        /* parent: wait for child to complete */
        wait(NULL);
        printf("Parent: child has finished\n");
    }
    return 0;
}
```

`execlp(file, arg0, arg1, ..., NULL)` – searches for file in PATH and executes it

Lab: Comparing the exec() Family

The exec() family has several variants:

Function	Argument Passing	Path Search	Environment Variables
`execl()`	list (l)	full path	inherited
`execlp()`	list (l)	PATH search (p)	inherited
`execle()`	list (l)	full path	specified (e)
`execv()`	array (v)	full path	inherited
`execvp()`	array (v)	PATH search (p)	inherited
`execve()`	array (v)	full path	specified (e)

Lab: Detailed Behavior of wait()

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child: working...\n");
        sleep(2);
        exit(42);  // exit with status 42
    } else {
        int status;
        pid_t child_pid = wait(&status);

        if (WIFEXITED(status)) {
            printf("Child %d exited with status %d\n",
                   child_pid, WEXITSTATUS(status));
        }
    }
    return 0;
}
```

Lab: How a Simple Shell Works

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[256];

    while (1) {
        printf("myshell> ");
        if (fgets(command, sizeof(command), stdin) == NULL)
            break;
        command[strlen(command) - 1] = '\0';  // remove newline

        if (strcmp(command, "exit") == 0)
            break;

        pid_t pid = fork();
        if (pid == 0) {
            execlp(command, command, NULL);
            perror("Command not found");
            return 1;
        } else {
            wait(NULL);  // wait for child to terminate → this is the shell's basic operation!
```

Lab: Shell Operation = Repeated fork() + exec() + wait()

```
Shell (Parent Process)
    │
    ├── Display prompt "myshell> "
    │
    ├── Read user input (e.g., "ls")
    │
    ├── fork() → create child process
    │       │
    │       └── [child] execlp("ls", "ls", NULL)
    │                   → execute ls
    │                   → exit()
    │
    ├── [parent] wait() → wait for child to terminate
    │
    ├── Confirm child termination
    │
    └── Loop back to the beginning for the next command
```

This is the core operating principle of UNIX shells (bash, zsh, etc.)!

Lab Key Summary

Function	Role	Key Characteristics
fork()	Creates a copy of the current process	Distinguishes parent/child by return value (parent: child pid, child: 0)
exec()	Replaces the process's memory image with a new program	Does not return to the original code on success
wait()	Waits for a child process to terminate	Collects child's exit status; prevents zombies
exit()	Terminates the process	Passes exit status to the parent

Combination patterns:

- `fork() + exec()` = run a new program

Part 5

Summary and Next Week Preview

Summary – Process Concept

- **Process** = a program in execution (a program is passive; a process is active)
- **Memory layout**: text (code) / data (global variables) / heap (dynamic allocation) / stack (function calls)
  - text and data are fixed in size; heap and stack grow dynamically
- **Process State**: New -> Ready -> Running -> Waiting -> Terminated
  - Only one process can be in the running state on a single core
- **PCB (Process Control Block)**: a data structure that stores all information about a process
  - state, PC, registers, scheduling info, memory info, accounting info, I/O info
- **Thread**: a unit of execution within a process; enables parallel execution on multicore systems (Ch 4)

Summary – Process Scheduling & Operations

Process Scheduling:

- Ready Queue / Wait Queue – processes move among various queues
- CPU Scheduler: selects a process from the ready queue and assigns it to the CPU
- **Context Switch**: save current process state + restore new process state (pure overhead)

Operations on Processes:

- **Process Creation**: parent -> child (distinguished by pid), forming a process tree
  - fork() = copy address space, exec() = load a new program
- **Process Termination**: exit() -> pass status to parent, collected via wait()
  - **Zombie**: terminated but wait() not called; **Orphan**: parent terminated first -> adopted by init/systemd
- **Android**: determines process termination order based on an importance hierarchy

Next Week Preview

Week 3 – Process 2

- Interprocess Communication (IPC)
  - Shared Memory
  - Message Passing
- Real-World IPC Implementations
  - POSIX Shared Memory, Mach Message Passing
  - Pipes (Ordinary / Named)
- Client-Server Communication
  - Sockets, Remote Procedure Calls (RPC)
- Lab: IPC implementation using pipes and shared memory

Textbook: **Ch 3, second half** (Sections 3.4 – 3.8)